Calcolatori Elettronici
Esercitazione 9

M. Sonza Reorda – M. Monetti
M. Rebaudengo – R. Ferrero
L. Sterpone – M. Duni
morgana.duni@polito.it

Politecnico di Torino
Dipartimento di Automatica e Informatica

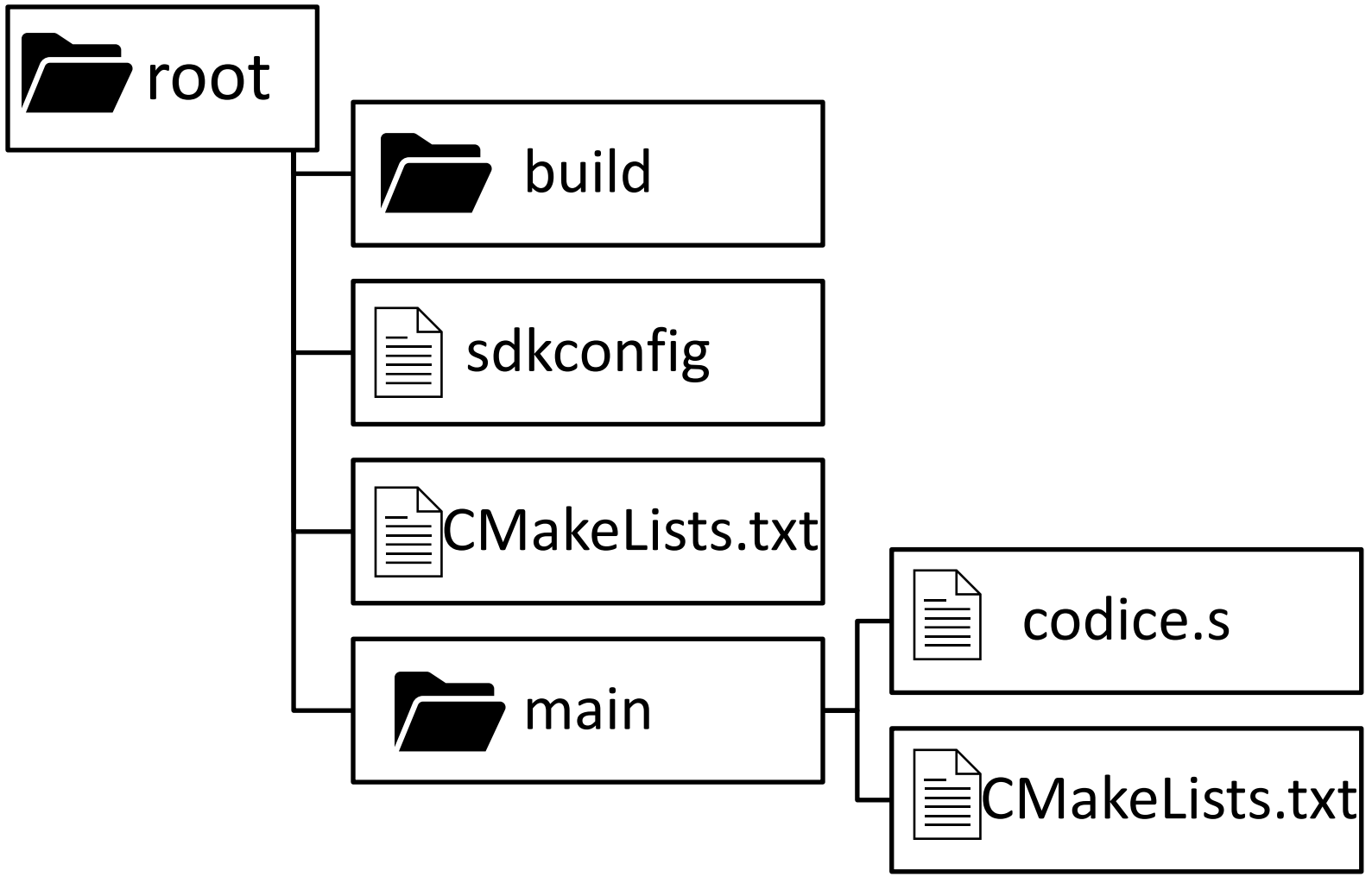
Obiettivi

- Programmazione della scheda ESP-32
- Interfaccia GPIO
- Accensione e spegnimento del LED

ESP-IDF

- ESP-IDF (Espressif IoT Development Framework) è il framework ufficiale e open source di Espressif Systems per lo sviluppo di firmware sui chip della serie ESP32.
- Installer online: https://dl.espressif.com/dl/esp-idf/
- Comandi per la shell ESP-IDF x.x CMD:
  - `idf.py set-target esp32c3` configura il progetto per il chip ESP32-C3
  - `idf.py build`: compila il progetto (crea la cartella build)
  - `idf.py fullclean`: cancella la cartella build, necessario per una nuova compilazione
  - `idf.py flash`: carica il firmware nella memoria della scheda. Opzionalmente si può indicare la porta COM, ad es. `-p COM4`

Struttura di un progetto ESP-IDF



Struttura di un progetto ESP-IDF

- La directory *root* può avere qualunque nome
- La directory *build* e il file *sdkconfig* sono generati automaticamente durante la compilazione
- Il file *CMakeLists.txt* nella directory root deve essere:

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(nome_directory_root)
```

- Il file assembly nel *main* può avere qualunque nome
- Il file *CMakeLists.txt* nella directory *main* deve essere:

```
idf_component_register(
    SRCS "nome_file_assembly.s"
)
```

Registri GPIO

- I registri GPIO sono memory-mapped con indirizzo base GPIO_BASE = 0x60004000
- GPIO_OUT_REG (offset 4): memorizza il latch software dell'output (cosa la CPU ha scritto in output)
- GPIO_OUT_W1TS_REG (offset 8): mettendo un bit a 1, setta il corrispondente pin a 1 (write 1 to set)
- GPIO_OUT_W1TC (offset 12): mettendo un bit a 1, setta il corrispondente pin a 0 (write 1 to clear)
- GPIO_ENABLE_REG (offset 20): imposta i pin come output
- GPIO_ENABLE_W1TS (offset 24): mettendo un bit a 1, imposta il corrispondente pin come output.

https://documentation.espressif.com/esp32-c3_technical_reference_manual_en.pdf

Esercizio 1

- Realizzare un programma che faccia lampeggiare il LED della scheda ESP32-C3, alternando accensione e spegnimento a intervalli regolari.
- Il LED è collegato al pin 8 (GPIO8).
- Dopo aver impostato lo stato del pin (1 acceso, 0 spento), attendere un certo delay (ad esempio 2000000 colpi di clock).
- Il programma dura indefinitamente (è all'interno di un ciclo infinito).

Esercizio 2

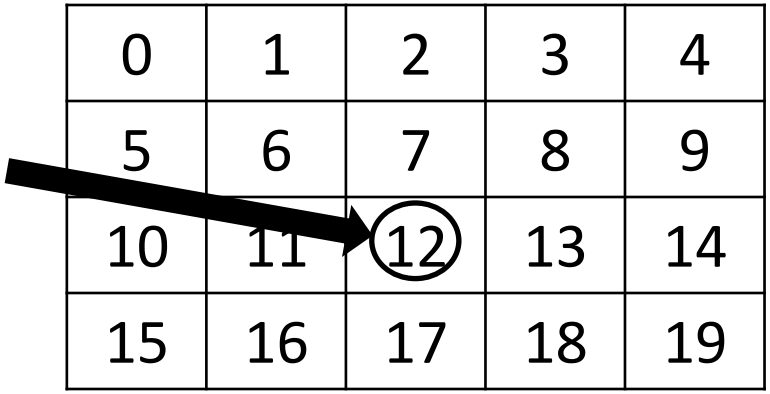
- Si vuole realizzare una macchina a stati finiti (FSM) basata su GPIO memory-mapped.
- GPIO6 e GPIO7 devono essere configurati come output.
- All'interno di un ciclo infinito, ogni X iterazioni deve essere invertito (toggle) il valore logico di GPIO6.
- Ogni Y iterazioni invertire il valore logico di GPIO7.
- Ogni Z iterazioni si determina lo stato della FSM leggendo il registro GPIO_OUT_REG:
  - stato 00: il LED deve essere spento
  - stato 10 o 11: il LED deve essere acceso
  - stato 01: il LED cambia stato (se spento, diventa acceso; se acceso diventa spento) in modo da mostrare un lampeggiamento. Lo stato del LED deve essere ottenuto leggendo GPIO_OUT_REG.

Esercizio 2 (continua)

- Testare differenti valori di X, Y, Z in modo che i tempi di accensione, spegnimento e lampeggiamento del LED siano ben visibili.
- Ad esempio, X = 3000000, Y = 5000000, Z = 300000.

Esercizio 3

- Sia data una matrice di byte, contenente numeri unsigned
- Si scriva una procedura **`contaVicini`** in grado di calcolare (e restituire come valore di ritorno) la somma dei valori contenuti nelle celle adiacenti ad una determinata cella.
- La procedura **`contaVicini`** riceve i seguenti parametri:
  - indirizzo della matrice
  - numero progressivo della cella X, così come indicato nell'esempio a fianco
  - numero di righe della matrice
  - numero di colonne della matrice.



Esercizio 3: esempio main

```
.equ RIGHE, 4
.equ COLONNE, 5
.data
matrice: .byte 0, 1, 3, 6, 2, 7, 13, 20, 12, 21, 11, 22, 10, 23,
9, 24, 8, 25, 43, 62

.text
main:   la a0, matrice
        li a1, 12
        li a2, RIGHE
        li a3, COLONNE
        jal contaVicini
        li a7, 10
        ecall
```

0	1	3	6	2
7	13	20	12	21
11	22	(10)	23	9
24	8	25	43	62

il valore restituito è 166, pari a
13 + 20 + 12 + 22 + 23 + 8 + 25 + 43

Esercizio 4

- Il gioco della vita sviluppato dal matematico John Conway si svolge su una matrice bidimensionale.
- Le celle della matrice possono essere vive o morte.
- I vicini di una cella sono le celle ad essa adiacenti.
- La matrice evolve secondo le seguenti regole:
  - una cella con meno di due vicini vivi muore (isolamento)
  - una cella con due o tre vicini vivi sopravvive alla generazione successiva
  - una cella con più di tre vicini vivi muore (sovrappopolazione)
  - una cella morta con tre vicini vivi diventa viva (riproduzione).
- L'evoluzione avviene contemporaneamente per tutte le celle.

Esercizio 4 (continua)

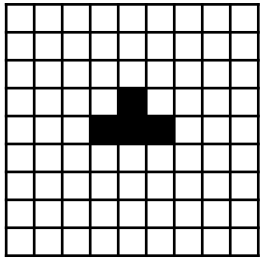
- Si scriva un programma in grado di giocare al gioco della vita.
- Il programma esegue un ciclo di N iterazioni; ad ogni iterazione chiama la procedura **`evoluzione`** che determina il nuovo stato delle celle nella matrice.
- La procedura **`evoluzione`** riceve i parametri:
  1. indirizzo di una matrice di byte, le cui celle hanno solo due valori: vivo (1) e morto (0)
  2. indirizzo di una seconda matrice di byte non inizializzata di pari dimensioni
  3. numero di righe delle due matrici
  4. numero di colonne delle due matrici.

Esercizio 4 (continua)

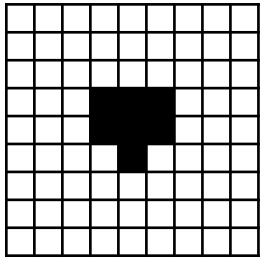
- La procedura **`evoluzione`** effettua un ciclo su tutte le celle della prima matrice:
  - per ogni cella chiama la procedura **`contaVicini`**, implementata nell'esercizio precedente, per contare il numero di vicini
  - in base allo stato della cella e al suo numero di vicini, setta lo stato futuro della corrispondente cella nella seconda matrice.
- Al termine del ciclo, la procedura **`evoluzione`** chiama la procedura **`stampaMatrice`** che visualizza a video la seconda matrice, passando i seguenti parametri:
  1. indirizzo della matrice
  2. numero di righe della matrice
  3. numero di colonne della matrice.

Esercizio 4: esempio

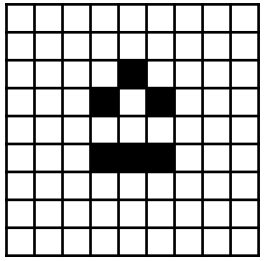
generazione 0



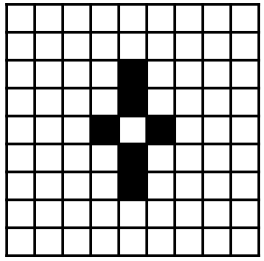
generazione 1



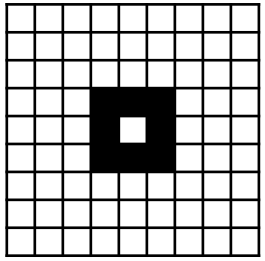
generazione 2



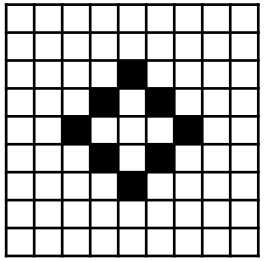
generazione 3



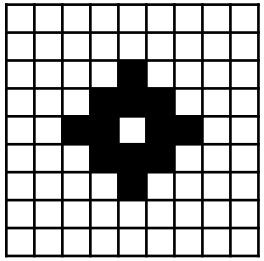
generazione 4



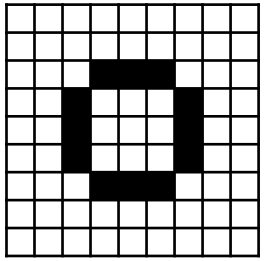
generazione 5



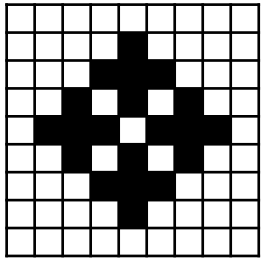
generazione 6



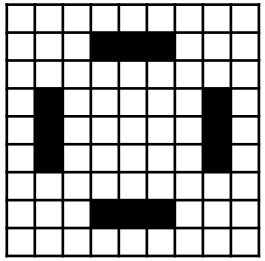
generazione 7



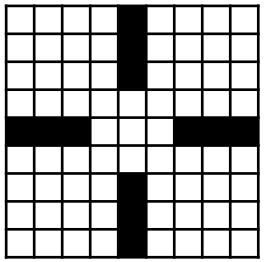
generazione 8



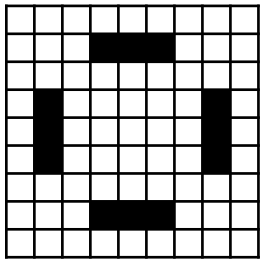
generazione 9



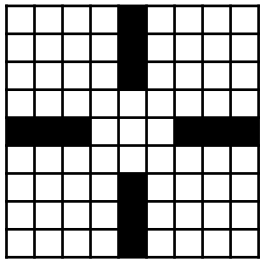
generazione 10



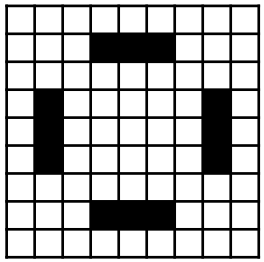
generazione 11



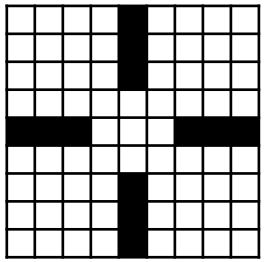
generazione 12



generazione 13



generazione 14



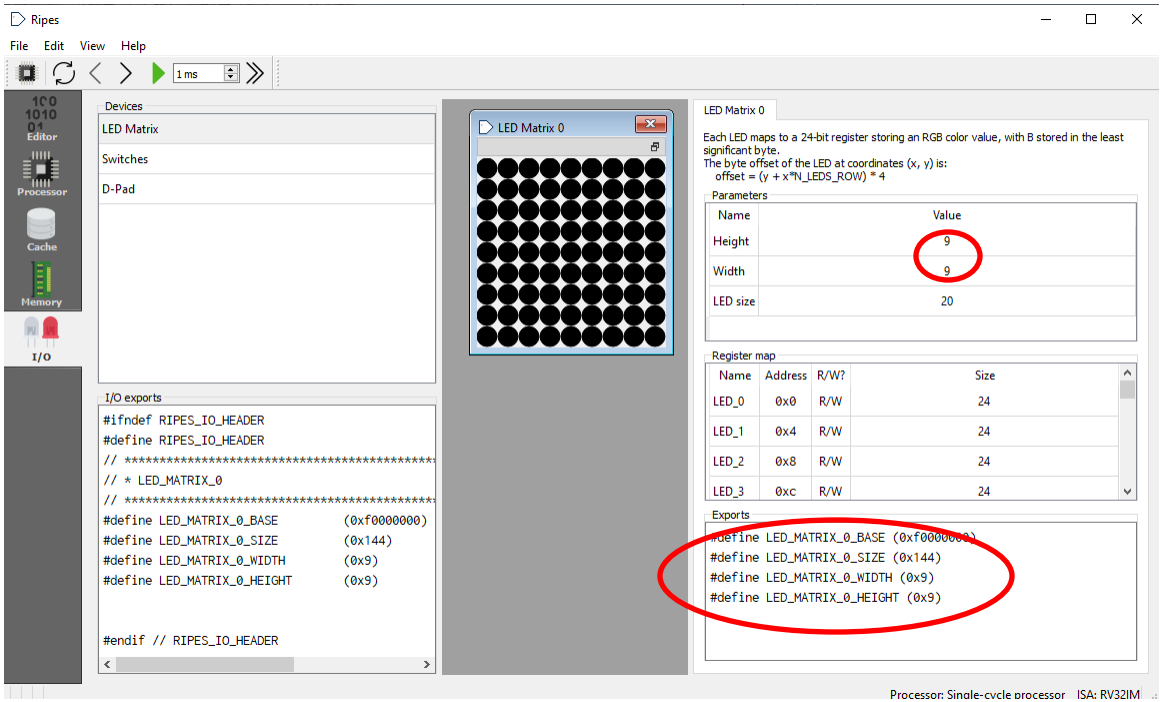
Per testare altre configurazioni: https://playgameoflife.com/

Esercizio 5

- Si scriva una procedura **`visualizzaMatriceLED`** che rappresenti la matrice calcolata dalla procedura **`evoluzione`** sui LED di Ripes.
- Dopo ogni visualizzazione, la procedura attende un certo delay (ad esempio 100000 iterazioni).
- La procedura riceve i seguenti parametri:
  1. indirizzo della matrice da visualizzare
  2. indirizzo del registro associato al primo LED
  3. numero di righe della matrice
  4. numero di colonne della matrice.

Esercizio 5 (continua)

- Per creare la matrice di LED in Ripes, spostarsi nella tab I/O e cliccare due volte su LED Matrix.
- Nella colonna a destra, modificare i parametri Height e Width per avere il corretto numero di LED
  - devono corrispondere alle dimensioni della matrice nel programma



Esercizio 5 (continua)

- Dopo aver creato la matrice di LED, il programma può accedere a 3 costanti:
  - LED_MATRIX_0_BASE: indirizzo del registro associato al primo LED
  - LED_MATRIX_0_WIDTH: numero di colonne della matrice
  - LED_MATRIX_0_HEIGHT: numero di righe della matrice.
- Ad ogni LED è associato un registro di 32 bit, di cui solo i primi 24 bit sono significativi.
- Per cambiare lo stato di un LED, occorre scrivere il codice del colore RGB nei 24 bit meno significativi del registro (esempio: 0 per nero e 0xFFFFFF per bianco).